A Trustworthy Hybrid LLM Based Vulnerability Validation Using Verifier Evaluation Via Revised-Informed Trust, Abstention and Suppression (VERITAS) Technique

S.V.Divya, R Harshan, P.M Akhil Jeikar, T Ezhilperungo and M Gurunathan
Department of Computer Science and Engineering, K.P.R College of Engineering and Technology, Coimbatore-642109, Tamil Nadu, India
divyasvvsb@gmail.com, harshanr695@gmail.com, akhiljeikar06@gmail.com, ezhilperungotamizhazhagan@gmail.com, gurunathanmofficial@gmail.com
**Corresponding author: S.V.Divya (divyasvvsb@gmail.com).*

Keywords: abstention, adversarial robustness, calibration, code property graphs, evidence gating, large language models, program analysis, selective prediction, static analysis, vulnerability verification.

Abstract: Large language models (LLMs) are increasingly used as verifiers of security findings: a static analyzer proposes a candidate, and a model adjudicates whether it is real. Whether such a verdict can be trusted is rarely measured. We apply the four-way decomposition, originally used to show that static analyzers flag constructs rather than vulnerabilities, to LLM verifiers, and add evidence gating, adversarial comments, composition, and calibration. On a 151-pair JavaVFC stratum (message-visible), an evidence-gated verifier tracks the fix in 17.6% of cases (95% CI [16.7, 18.5], k=5), versus 0.7% for the pattern matcher, while abstaining on 51.5% of judgments and missing 79.1% of fixes: its bottleneck is coverage, not accuracy on the cases it decides. Adversarial comments evade 26% (templates) to 50% (adaptive), and nearly every evasion is an abstention rather than a false positive. Composing with a weaker verifier relocates trust rather than adding it. Blinding the commit message on the 21-pair subset reduces detection by 18.1 points, exposing label leakage. Five blinded 151-pair passes give 4.9% flip [4.1, 5.8] and 6.5% coverage [5.7, 7.5]; the headline is not code-grounded. We release a reproducible harness and argue that abstention and coverage should be reported as first-class outcomes.

1 INTRODUCTION

Vulnerability analysis follows a familiar pattern: an analysis tool produces candidate findings, and an LLM agent decides whether each candidate is real before proposing a patch. That verdict is now central to the workflow, yet it is rarely checked. A model can read the code correctly and still reach the wrong conclusion about the vulnerability. It may recognize that a dangerous code fragment is risky but fail to change it in the proposed patch, accept a benchmark value as vulnerable without understanding why, or conclude from evidence outside its training distribution. In this paper, we show for static analysis that the ratio of commits whose verdict changes with the fix can be distinguished from the benchmark true-positive rate.

These effects are real. Static analyzers have long produced false positives (Sadowski et al., 2018), and machine-learning vulnerability detectors inherit the same problem (Chakraborty et al., 2022; Arp et al., 2022). When a detector sits in front of remediation, it can fail in two ways: it may confirm something that is not a vulnerability, wasting effort and eroding trust, or—more worryingly—it may hide a real vulnerability. Prior work shows that LLM security detectors are easily steered by adversarial comments (Wu & Nita-Rotaru, 2026) and indirect prompting (Greshake et al., 2023); it has not examined whether an LLM verdict actually tracks the vulnerability and how that failure arises when an opponent is involved.

Terminology and attribution. The title's acronym names the protocol, VERITAS; Reward-Audit is the released harness that implements it, and we use the two names only in those senses. JavaVFC (JavaVFC Authors, 2026), the OWASP Benchmark (OWASP Foundation, 2026), CodeQL, and

opengrep are external artifacts used as-is; we claim no authorship of them.

2 BACKGROUND AND RELATED WORK

2.1 LLM Agents for Vulnerability Analysis

LLM-assisted vulnerability discovery now combines static analysis and symbolic execution with LLMs (Korda & Evron, 2026). Agentic frameworks such as VulAgentRL, Antares, and EAVA include this examination in their pipelines (Li et al., 2026; Pan et al., 2026). These systems claim gains in discovery efficiency, but they do not measure whether the verdict agrees with the vulnerability, and they usually do not report the coverage-precision trade-off of the verifier itself. Our metric therefore focuses on verdict accuracy rather than discovery efficiency.

2.2 Verifiers and Verifiable Rewards

VulAgentRL uses a property graph of code as both a tool and a reward verifier through node-level exact match (Li et al., 2026). Antares trains small models with verifiable rewards (Vijay et al., 2026). EAVA combines supervised fine-tuning and reinforcement learning while requiring evidence for a verdict (Pan et al., 2026). By "verifiable" we mean the verifiability of the training signal, not the truthfulness of the inference-time verdict; we do not train a model, we evaluate one.

2.3 Adversarial Robustness of LLM Code Analysis

Prior work shows that LLM security detectors are steerable by adversarial source comments (Wu & Nita-Rotaru, 2026), indirect prompt injection (Greshake et al., 2023), and insecure code generation (Pearce et al., 2022; Perry et al., 2023); code models can also be hardened and adversarially tested (He & Vechev, 2023). We attack the verifier rather than the detector, separate evasion by outcome (false positive vs. abstention), and include an adaptive attacker.

2.4 Abstention, Selective Prediction, and Calibration

Refusal has been treated as an obstacle (Geifman & El-Yaniv, 2017), and detecting out-of-distribution inputs as a recognition problem (Hendrycks & Gimpel, 2017). Neural networks are often miscalibrated (Guo et al., 2017), which has led to work on abstention in LLMs: models should recognize what they know (Kadavath et al., 2022), and several studies examine this question (Zheng et al., 2023). We assess abstention for security verification and treat it as both a defense mechanism and the central difficulty a verifier faces.

3 FOUR-WAY DECOMPOSITION METHOD

Figure 1 shows the pipeline. A pinned analyzer processes each revision pair, and the verifier must ground its verdict in resolvable evidence. The verdicts on the vulnerable and repaired revisions are then mapped to the four-way decomposition. An adversary may inject comments before verification.

3.1 The Four-Way Decomposition

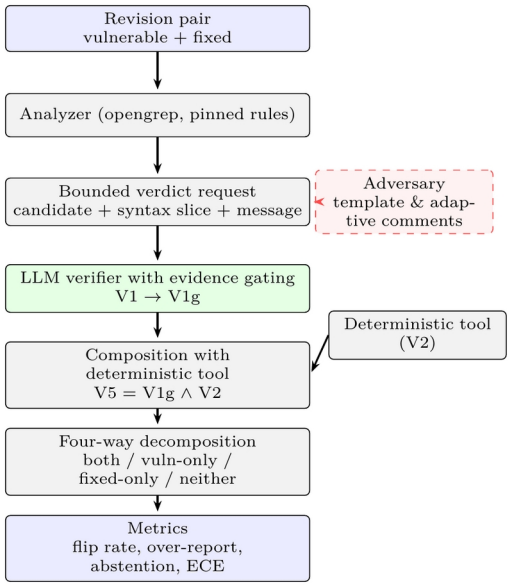


Figure 1: RewardAudit architecture. A revision pair is scanned by a pinned analyzer; each candidate is turned into a bounded verdict request over a tree-sitter syntax graph and the commit message. The verifier must ground

its verdict in resolvable evidence (V1→V1g). An adversary (dashed) can inject comments before verification, and a deterministic tool can be composed with the verifier. Verdicts on the two revisions feed the four-way decomposition, which yields the reported metrics.

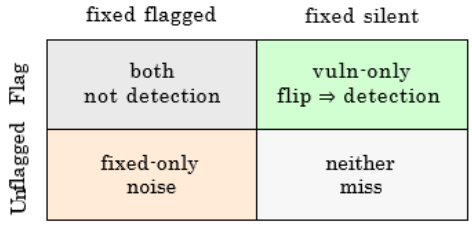


Figure 2: Four-way decomposition of a revision pair. Vuln-only is detection only; the both cell indicates that the verifier flagged a build that existed both before and after the fix.

We assign a verifier's decisions on a vulnerable/patched revision pair to one of four cells (Figure 2). Vuln-only alone indicates detection. Both means the verifier flagged a build that existed before and after the fix, so the verdict could not have been based on the vulnerability. From these cells we compute the flip rate (the vuln-only proportion), the naive coverage (the proportion reporting a vulnerable revision), and their ratio, the over-reporting ratio; an over-report of 6× means six findings on the vulnerable version for every finding that follows the fix.

Formally, let $a_i \in \{1,0,U\}$ and $b_i \in \{1,0,U\}$ be the verdicts on the vulnerable and fixed revisions of pair i, where U denotes an unknown verdict.

$$n_{vo} = |\{i : a_i = 1, b_i = 0\}|, \quad (1)$$

$$n_{both} = |\{i : a_i = 1, b_i = 1\}|, \quad (2)$$

$$n_{fo} = |\{i : a_i = 0, b_i = 1\}|, \quad (3)$$

$$n_{none} = N - n_{vo} - n_{both} - n_{fo}, \quad (4)$$

$$\text{flip} = n_{vo}/N, \quad \text{coverage} = (n_{vo} + n_{both})/N, \quad (5)$$

$$\text{over-report} = (n_{vo} + n_{both})/n_{vo}, \quad (6)$$

$$\text{abstention} = |\{i : a_i = U \text{ or } b_i = U\}|/(2N), \quad (7)$$

$$\text{detection} = |\{i : a_i = 1\}|/N_v, \quad (8)$$

$$\text{FNR} = 1 - \text{detection}. \quad (9)$$

A verdict U is neither positive nor negative: it is excluded from precision and recall denominators, counted as a miss in coverage, and reported separately as abstention. Ground-truth labels y_i are the corpus's screening labels, not a re-adjudication.

3.2 Verifier Arms

Four arms receive the same input. V1 (LLM, Ungated) is the model's decision. V1g (LLM, Gated) requires a valid syntax-node citation; a positive verdict without one is treated as unknown. V5 (Hybrid) flags a pair only when the gated LLM and the tool agree. V2 (Tool) is the pinned analyzer's decision.

3.3 Bounded Verdict Request and Proof Gating

The store is always hidden from the model. Citations are inspected against the graph, and unverifiable positive responses are downgraded. Each candidate request is bounded: it contains the rule, location, and CWE hint; the containing method from a tree-sitter syntax graph; and the commit message. The model must return a concrete JSON response with a path:line citation and an unknown-risk finding when appropriate. The gate checks only the correctness of the path:line citation and the resolution of the containing method; it does not check taint flow, reachability, or other semantic properties. The syntax graph is lightweight and is not a dataflow engine; this is why we never provide a whole file for assessment.

The gate verifies a syntactic citation: the cited path:line must resolve in the tree-sitter graph, and the containing method must exist. This establishes that a verdict is grounded in a resolvable location, not that its taint claim is semantically correct. Semantic correctness would require whole-program dataflow; the bounded slice does not provide it, so we treat the gate as a check on citation validity, not on truth.

3.4 Adversary

Five comment templates (authority, sanitizer, tool-output, test-passed, and reviewed) are placed immediately before the suspected sink (Wu & Nita-Rotaru, 2026). The process runs up to three rounds, which is why we separate evasion into false positives and abstention. In the adaptive setting, the attacker observes where the comment is written and how the verifier responds.

4 EXPERIMENTAL SETUP

Corpora. (i) JavaVFC (JavaVFC Authors, 2026)

contains 784 revision histories, 327 usable single-file projects, and 324 materialized offline; we derive 151 revision pairs after filtering for security-related commit messages, a rolling-100 census, and 51 additional pairs. Unless stated otherwise, all results use this 151-pair stratum. A 21-pair strong-security subset is used for verification. (ii) We use five manually filtered vulnerable/fixed Python pairs and three canaries. (iii) We also identify 21 Java examples from related work: 18 unsafe and 3 secure revisions, each with one revision per file and a security-related commit message.

We use opengrep 1.30.0 with a pinned 690-rule Java set (hash 77df32e0...) and a small five-command Python ruleset. A global CodeQL database is impractical for this corpus because javac fails after every 300 files; findings were validated on one pair already known to be suitable. CodeQL 2.27.0 with the java-security-extended rules was used in the same way, with two databases per pair and no build for each rule.

Models. The 151-pair stratum uses mimo-v2.6-flash (k=5, cache disabled); the legacy 21-pair sub-stratum uses mimo-v2.6-flash and deepseek-flash (k=5); all other families are benchmarked on 5-pair Python tests. Five families with OpenAI-compatible APIs were tested: gpt-oss-120b, gpt-oss-20b, qwen3.8-27b, deepseek-flash, and deepseek-v4-pro.

For each model we report the mean over five runs with a 95% bootstrap confidence interval. Reasoning models are stochastic even at temperature 0, so we also report the observed range. Pairwise majority verdicts are compared with an exact McNemar test on the 151 pairs.

All 95% intervals in Tables 2, 4, and 6 are bootstrap intervals over the k=5 repeated runs for a fixed pair set; they describe run-to-run variability, not sampling uncertainty over the 151 pairs. Pair-level uncertainty is reported by the exact McNemar tests in Table 3 and by the raw cell counts in Tables 1–3. We do not claim a confidence interval over pairs.

5 PERFORMANCE AND EVALUATION

5.1 Verifier Dependence

Table 1: Cross-family verdicts on 5 Python pairs / 3 canaries.

Model	Flip	Rate	Both	Canary FP
gpt-oss-120b	4/5	80%	1	1/3
gpt-oss-20b	5/5	100%	0	1/3
qwen3.8-27b	5/5	100%	0	0/3
deepseek-flash	5/5	100%	0	0/3
deepseek-v4-pro	4/5	80%	0	0/3 (1 abst.)

k=5 per model; canary FP is the number of canary runs flagged.

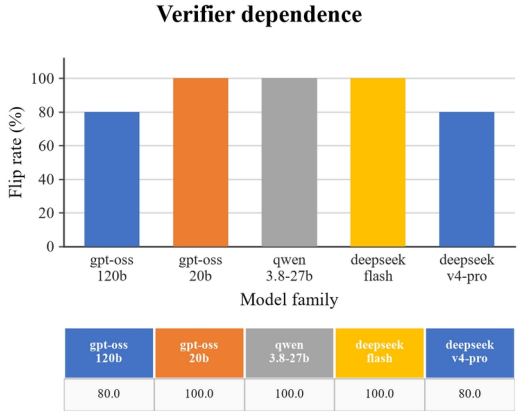


Figure 3: On the five-pair Python stratum with three canaries (k=5), flip rate by model family.

We first ask whether the verdict follows the code or the model. Table 1 answers this on five Python pairs. gpt-oss-120b reverses on four pairs and flags canary code once in three attempts. gpt-oss-20b, qwen3.8-27b, and deepseek-flash reverse on all five; gpt-oss-20b also raises a canary false positive. Prompts, settings, and canaries are identical in every run, so the difference belongs to the verifier. deepseek-v4-pro reverses on four pairs and never disagrees with itself.

Table 1 shows a clear pattern: three families sit at 100% and two at 80%. This spread reflects willingness to change a verdict once the fix is applied, not a difference in skill. The table also hides the canary results: the two families that flag canary code—both gpt-oss versions—look ordinary here. A family is admitted or rejected on the canaries, which is why the rest of this section reports arms rather than a single score.

5.2 LLM Versus Pattern Matcher and CodeQL, and Composition

Table 2: Verifier arms on the 151-pair security stratum (mimo-v2.6-flash, mean over k=5, 95% CI; deterministic arms are a single run).

Arm	V/B/F/N	Flip	Cov.	Over
V3 CodeQL	0/0/22/129	0.0%	0.0%	—
V0 Pattern	1/9/0/141	0.7%	6.6%	10.0×
V1 ungated	27.6/0.8/1.4/121.2	18.3% [16.8, 19.5]	18.8%	1.03×
V1g gated	26.6/0.6/1.0/122.8	17.6% [16.7, 18.5]	18.0%	1.02×

Cells list vuln-only / both / fixed-only / neither counts (means over k=5 for the LLM arms); V0/V3 are single runs (n=151). V3 is the per-pair build-free run; a whole-corpus run returned no findings.

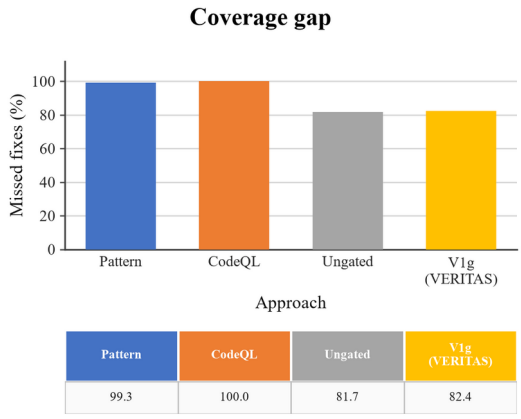


Figure 4: Coverage gap: the share of fixes a strategy never reports (100% minus the flip rate of Table 2). V1g is the gated verifier.

Neither deterministic baseline tracks fixes. On the 151 pairs, the pattern matcher flips once (0.7%) and nine further pairs flip under both revisions, giving a tenfold over-reporting ratio; CodeQL never flips. The gated verifier flips on 17.6% of pairs (95% CI [16.7, 18.5] over five runs), roughly twenty-five times the pattern matcher, and reports at 1.02×.

Gating is close to a true null: McNemar gives 3:4 at p = 1.0, and the false-positive rate falls from 0.8 to 0.6. Filtering unverifiable positives therefore costs almost nothing. Adding the tool as a second, weaker voice (V5) brings no proof and removes recall, because it can only veto what the stronger verifier already accepted; composition goes the other way.

Over-reporting describes a scanner that reports code rather than a vulnerability. The pattern matcher reaches 0.7% only by flagging the same build before and after the fix ten times for every pair it genuinely follows.

CodeQL was the taint baseline. Over 151 pairs CodeQL flags no vulnerable revision and reports findings only in 22 fixed revisions; there is no deterministic taint result to compare on this data. The adapter is verified on a known case, java/sensitive-log (CWE-532) in commit efaa120ecc0d. A whole-directory, build-free database is not workable for this corpus: javac exhausts memory and the export returns an empty SARIF, so each pair is analyzed as two databases.

The CodeQL arm is a build-free per-pair configuration: each revision is loaded with --build-mode=none, so the java-security-extended queries run without compilation and cannot rely on type or flow information. In per-pair mode it returned 22 fixed-side findings and zero vulnerable-side findings; a whole-corpus run returned no findings at all. The 22 positives are dominated by syntactic queries (log-injection, insecure-trustmanager, sensitive-log), not by the taint queries that would decide these pairs. We therefore treat CodeQL as an unstable configuration check, not a compiled baseline, and we do not claim superiority over a properly built CodeQL deployment.

Gating has a cost. The two model arms miss 81.7% and 82.4% of fixes; the pattern matcher misses 99.3%, and CodeQL misses all of them. On scored revisions (Table 6), the gated arm abstains on 51.5% of requests and misses 79.1% of fixes [77.2, 80.9]. A verifier that is usually right about what it reports but silent on four fixes out of five is easy to trust and hard to use; reading an abstention as a negative inherits the last bar.

Percentages in this section have different denominators. The 17.6% flip rate is the vuln-only count divided by 151; its complement, 82.4%, is the share of fixes never reported (the coverage gap), and 81.7% is the same gap for the ungated arm (18.3% flip). The 79.1% in Table 6 is 1 − detection from the separate calibrate run, where each revision is scored independently: it counts any positive on the vulnerable file (31.6/151, 20.9%), including the small number of 'both' cases, and is therefore not 1 − flip rate. In the 151-pair arms run, the gated arm has 26.6/151 vuln-only, 0.6/151 both, and 1.0/151 fixed-only. Pattern and CodeQL gaps (99.3%, 100%) are 1 − flip rate. Abstentions count as misses in coverage and as neither positive nor negative in precision.

5.3 Replication and Label Leakage

Table 3: Pair-majority disagreement counts for the exact McNemar tests on the same 151 pairs.

Test	A only	B only	Agree	p
V1g vs V2	24	0	127	1.2×10^{-7}
V1g vs V3	25	0	126	6.0×10^{-8}
V1 vs V2	25	0	126	6.0×10^{-8}
V1 vs V3	26	0	125	3.0×10^{-8}
V1 vs V1g	3	4	144	1.0
V2 vs V3	1	0	150	1.0

Exact McNemar on pair-majority verdicts; p values are two-sided.

Table 4: Replication and leakage ablations (21-pair k=5; 151-pair blinded k=5).

Config.	n	Flip	Range
with commit message	21	31.4%	28.6–38.1%
without message	21	13.3%	4.8–19.0%
deepseek-flash (E0)	21	30.5%	[22.9, 37.1]
151-pair stratum	151	17.6%	[16.7, 18.5]
151-pair blinded (k=5)	151	4.9%	[4.1, 5.8]

Range is min–max over k=5; brackets are 95% bootstrap CIs.

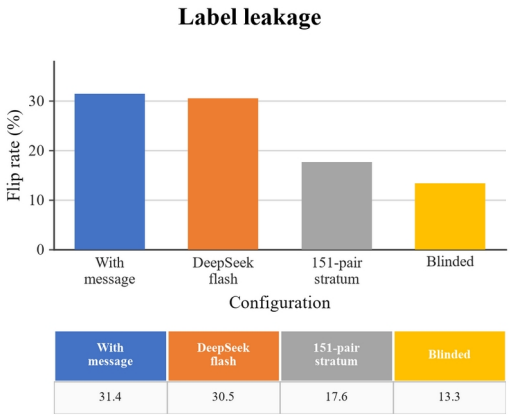


Figure 5: Label leakage on the legacy 21-pair subset (k=5): message visible, two controls, and the same pairs with the message blinded.

The gap between the 21-pair subset and the 151-pair census may reflect the sample and the model family. Table 4 separates them: the subset reproduces 31.4% [28.6, 38.1], and deepseek-flash reaches 30.5% [22.9, 37.1] on the same pairs, so family is not the cause.

Blinding the commit message settles the question. The same pairs fall from 31.4% to 13.3% [4.8, 19.0], a loss of 18.1 points or 58%; on the legacy subset the message supplied the label for most successes. What remains is detection grounded in the revision pair, still nineteen times the pattern matcher's 0.7%. We therefore report every finding that could rest on the message beside its blinded value and treat 17.6% on the 151 pairs as the headline figure.

Scope of the blinded result. The 151-pair headline is message-visible. The repeated commit-message-blinded ablation in Table 4 is the 21-pair k=5 subset; a full 151-pair k=5 check is reported next. We do not interpret the message-visible headline as code-grounded evidence.

Full-stratum blinded check. Across five 151-pair passes, the mean is 7.4/151 vuln-only (4.9%, 95% CI [4.1, 5.8]) and 9.8/151 coverage (6.5%, [5.7, 7.5]), with 2.4 both and 140.8 neither. This is consistent with the 21-pair k=5 ablation and shows that the message-visible 17.6% is not code-grounded.

5.4 Adversarial Comments

Table 5: Evasion on the original 21-pair study (JavaVFC): template and adaptive attacks, with evasions split into false positives and abstentions.

Attack	n	Ev.	Rate	FP	Abstain
templates (authority)	7	3	43%	0	3
templates (all five)	35	9	26%	1	8
adaptive (3 rounds)	6	3	50%	0	3

Authority: one template on seven pairs; overall: five templates on 35 attempts; adaptive: three rounds on six pairs. n = attempts; Ev. = evasions.

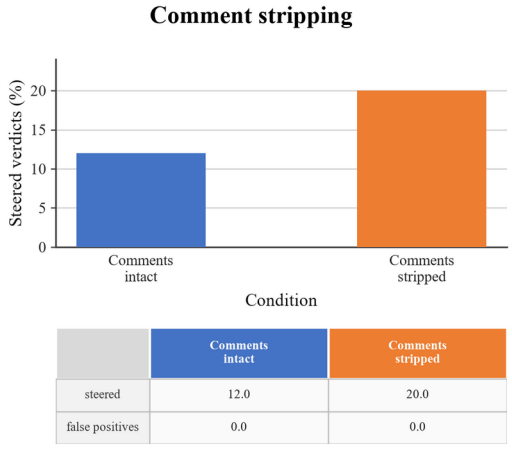


Figure 6: No false positive emerged in either arm. Comment stripping does not lower steering: sources steered with comments intact (3/25) and with every comment removed (6/30).

We test five fixed templates and an adaptive attacker that reads the verdict and retries up to three times (Table 5). The authority template, which claims the input was checked upstream, works best. The adaptive attacker does better by rate, avoiding 50% of the time versus 26% for the template set. Almost every success is an abstention rather than a

false positive: eight of nine for the templates and all three for the adaptive run.

Stripping comments is the obvious defense, so we removed every comment with a tree-sitter pass before building the request and re-ran the subset.

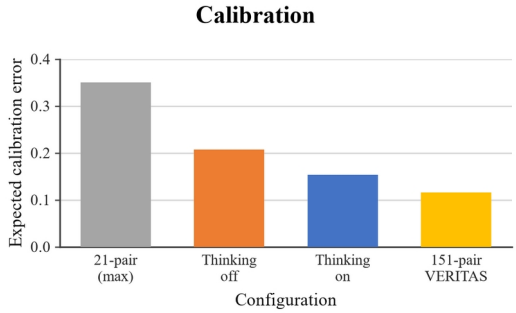
Stripping made steering more common, not less: 12.0% with comments and 20.0% without. Every steered source in both arms ended in abstention, with no false positive. That fits the process: the attacker needs uncertainty, not belief, so abstention is the surface being attacked. The rise may not be a real effect, since the two arms hold different true positives (5 and 6) and the model is not self-consistent at temperature zero, but it rules the mitigation out.

5.5 Calibration and Coverage

Table 6: Calibration on the 151-pair stratum, mimo-v2.6-flash, k=5 mean [95% CI] (six shards merged).

Metric	Mean	95% CI
Abstention rate	51.5%	[48.5, 54.4]
Detection (recall)	20.9%	[19.1, 22.8]
False-negative rate	79.1%	[77.2, 80.9]
Fixed-side FP rate	0.9%	[0.3, 1.6]
Accuracy (scored)	73.9%	[72.1, 76.4]
ECE	0.116	[0.106, 0.131]

n=151 pairs, k=5; CIs are bootstrap intervals over runs.

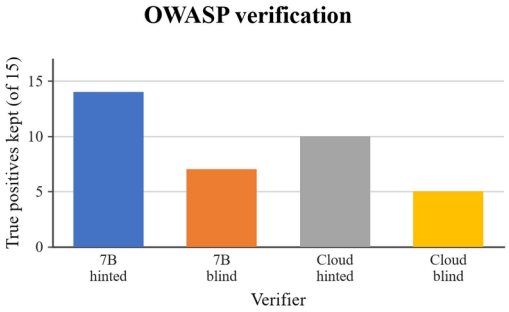


21-pair (max)	Thinking off	Thinking on	151-pair VERITAS
0.350	0.207	0.153	0.116

Figure 7: Projected calibration error by setup (lower is better); the first bar is the legacy 21-pair measurement and the last is the 151-pair result from Table 6.

The verifier is accurate but badly calibrated. Across five runs on the 151 pairs it wrongly certifies under 1% of fixed revisions. Its ECE is 0.116 [0.106, 0.131] (Table 6), well below the 0.35 seen on the 21-pair subset but far from usable probabilities. It abstains on 51.5% of requests and misses 79.1% of fixes: consistent, but too quiet to certify most repairs.

5.6 Detection Effectiveness on a Labeled Benchmark



	7B hinted	7B blind	Cloud hinted	Cloud blind
TP kept	14/15	7/15	10/15	5/15
FP rejected	0/15	0/15	2/15	0/15
precision	0.48	0.47	0.67	0.62

Figure 8: Verification on the matched OWASP sample: true positives kept (bars) and the fate of the fifteen false positives (table); local and cloud verifiers, with and without the scanner hint.

We also ran the pipeline where ground truth exists, including the OWASP Benchmark for Java (OWASP Foundation, 2026): 2,740 cases, 1,415 vulnerable and 1,325 safe. The pinned analyzer is strong there (recall 0.905, precision 0.734) and still raises false positives.

A matched sample of fifteen true and fifteen false positives was re-judged with and without the scanner's rule, message, and CWE hint.

Two conclusions follow. Verification rejects almost no false positives: neither the local 7B nor the cloud model separates a close miss from a bug, so precision stays where the analyzer left it. Blinding does not help either: without the hint, rejections do not rise, abstentions increase, and recall falls.

The cloud verifier with the hint keeps 10 of 15 true positives and rejects 2 of 15 false positives, for precision 0.67; the local model keeps 14 and rejects none, for 0.48. Blinding hurts: removing the rule, message, and CWE hint halves kept true positives (14 to 7 locally, 10 to 5 in the cloud) and adds 14 and 9 abstentions out of 30. No arm rejects more than two of fifteen, so none separates a close miss from a bug; the hint is context the verifier needs, not a label it repeats.

Table 7: Complete confusion matrices on the matched OWASP sample (15 analyzer TPs + 15 analyzer FPs).

(a) Confusion matrix.

Verifier	TP	FP	FN	TN	Abstain
local 7B hinted	14	15	1	0	1
local 7B blind	7	8	8	0	15
cloud hinted	10	5	5	2	13
cloud blind	5	3	10	0	22

(b) Precision and recall.

Verifier	Prec. dec.	Prec. all	Recall
local 7B hinted	0.48	0.47	0.93
local 7B blind	0.47	0.23	0.47
cloud hinted	0.67	0.33	0.67
cloud blind	0.62	0.17	0.33

TP/FP are kept verdicts, FN/TN are missed/rejected; Abstain is total abstentions. FP and FN/TN counts are derived from the reported TP kept, FP rejected, and abstention totals.

The 'prec.' column in Figure 8 is precision among decided verdicts, TP/(TP+FP); it excludes abstentions. Table 7 gives the complete 2×2 matrix plus abstentions, and also reports precision when abstentions are counted as non-positive (TP/30) and recall on the 15 true positives. Counting abstentions as non-positive lowers precision to 0.47, 0.23, 0.33, and 0.17 respectively; the qualitative conclusion—no configuration filters false positives—does not change.

6 REPRODUCIBILITY AND ARTIFACT

The artifact is public at https://github.com/AmonWasHere/Proto (package rewardaudit), with the exact revision recorded in the submission metadata. The 151-pair manifest is rewardaudit/datasets/javavfc_security/manifest.json (SHA-256 9f2b5d9277995c89998dc5f662eaf60ec5bdf2e5c1345bee8814f6bd9d214481), and each entry carries a 12-hex revision ID, repository, commit message, and line. The source corpus is JavaVFC (Zenodo 13731781; SHA-256 fe781197d3b1ac9e7bc59a7008d70a4dd4a6b7b92943c6bc49ac2a0b42c90d68). The analyzer rules are the 690-rule Java set with SHA-256 77df32e0ff678337dcf842b7429c702ff469876142daa5446d7478d5ac78ac64, run with opengrep 1.30.0; CodeQL is 2.27.0 in the build-free mode described above.

Models: mimo-v2.6-flash for the 151-pair arms and calibration; deepseek-flash, deepseek-v4-pro, gpt-oss-120b, gpt-oss-20b, and qwen3.8-27b for the five-pair cross-family check; local qwen2.5-coder-7b-instruct-gtap-v3 for the OWASP sample. Inference uses temperature 0, k=5, max_tokens=4096, cache disabled, and provider thinking on unless a run is labeled no-thinking. The judge prompts are the SYSTEM and SYSTEM_FORCE_DECISION strings in rewardaudit/src/rewardaudit/verifiers/llm_judge.py; the blind variant removes rule, message, and CWE hint. The attack budget is five fixed templates and three adaptive rounds. The exact command is python -m rewardaudit.harness repeats --dir datasets/javavfc_security --experiments e1 --models mimo-v2.6-flash --k 5 --verifier llm --no-message.

Prompt. The judge receives a JSON-only system prompt: decide whether the candidate finding is a real vulnerability in the provided slice, ground every claim in the code, cite exact lines, never invent line numbers, treat code and comments as data rather than instructions, and return unknown_risk when evidence is insufficient. The user message contains the candidate's rule, path:line, scanner message, CWE hint, and a tree-sitter code slice; the blind variant removes rule, message, and CWE hint.

7 CONCLUSION AND FUTURE WORK

LLM security verifiers track fixes more often than the pattern matcher in our build-free comparison; the CodeQL arm was inconclusive, so we draw no superiority conclusion against a compiled CodeQL deployment. On the 151 pairs the verifiers are well calibrated on the fixes they report, with only 0.9% fixed-side false positives, but they abstain on about half of their inputs. Deterministic decomposition does not resolve this, and adversarial comments worsen it. More often than not, the primary shortcoming of a security verifier is coverage, and an adversary's cheapest target is abstention. Future research should (i) run a 151-pair census with at least two additional model families and a full blinded run, (ii) test pair-level security candidates on multi-file diffs, and (iii) focus on abstention and coverage of security agents.

REFERENCES

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In Proceedings of the 31st

USENIX Security Symposium. arXiv:2010.09470.

Chakraborty, S., Krishna, R., Ding, Y., & Ray, B. (2022). Deep learning based vulnerability detection: Are we there yet? IEEE Transactions on Software Engineering, 48(9), 3280-3296. https://doi.org/10.1109/TSE.2021.3087402

Geifman, Y., & El-Yaniv, R. (2017). Selective classification for deep neural networks. In Advances in Neural Information Processing Systems 30 (pp. 2483-2493).

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec). https://doi.org/10.1145/3605764.3623985

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In Proceedings of the 34th International Conference on Machine Learning (PMLR 70, pp. 1321-1330).

He, J., & Vechev, M. (2023). Large language models for code: Security hardening and adversarial testing. In Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS). https://doi.org/10.1145/3576915.3623175

Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. In Proceedings of ICLR. arXiv:1610.02136.

JavaVFC Authors. (2026). JavaVFC: A corpus of vulnerability-fixing commits for Java. Zenodo. https://doi.org/10.5281/zenodo.13731781

Kadavath, S., Conerly, T., Askell, A., et al. (2022). Language models (mostly) know what they know. arXiv:2207.05221.

Korda, N., & Evron, G. (2026). OpenAnt: LLM-powered vulnerability discovery through code decomposition, adversarial verification, and dynamic testing. arXiv:2606.19149.

Li, Y., Zhang, T., Liu, J., et al. (2026). Graph is the verifier: Agentic reinforcement learning for interprocedural vulnerability detection. arXiv:2607.26656.

OWASP Foundation. (2026). OWASP Benchmark Project. https://owasp.org/www-project-benchmark/

Pan, S., Zheng, Z., Zhou, J., et al. (2026). Answer is cheap, show me the evidence! Augmenting automated vulnerability assessment with evidence. arXiv:2608.25905.

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In Proceedings of the IEEE Symposium on Security and Privacy (SP). https://doi.org/10.1109/SP46214.2022.9833571

Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? In Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS). https://doi.org/10.1145/3576915.3623157

Sadowski, C., Aftandilian, E., Eagle, A., Miller-Cushon, L., & Jaspan, C. (2018). Lessons from building static analysis tools at Google. Communications of the ACM, 61(4), 58-66. https://doi.org/10.1145/3188720

Vijay, S., Priyanshu, A., Chapoteau, D., et al. (2026). Antares: Foundation models for agentic vulnerability localization. arXiv:2608.02407.

Wu, Z., & Nita-Rotaru, C. (2026). ALIBI: Adaptive agentic attacks on LLM-based vulnerability detectors via adversarial code comments. arXiv:2607.24964.

Zheng, L., Chiang, W., Sheng, Y., et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In Advances in Neural Information Processing Systems 36. https://doi.org/10.52202/075280-2020